基于 nanoGPT 的文本生成实验及结构优化探究

Text Generation Experiment and Structure Optimization Based on nanoGPT

张清越

摘　要　本实验直接套用 GitHub 开源 nanoGPT 框架，以莎士比亚戏剧文本为数据集，完成模型训练与推理全流程验证，并围绕模型结构参数与归一化方式开展两项创新探究。实验成功构建参数量 0.8M 的字符级自回归模型，经 2000 次迭代训练后，训练损失降至 1.7648，验证损失为 1.8857，生成文本可准确复现戏剧对话格式。创新实验表明：block_size 从 32 增至 128 时，训练损失降低 4.3%，但单次迭代时间增加 75.6%；将 Pre-LN 改为 Post-LN 后，训练损失震荡幅度提升 30%，验证了归一化方式对训练稳定性的影响。实验完整复现了 nanoGPT 的核心流程，为大模型架构理解与参数调优提供了实践支撑。

关键词　nanoGPT；文本生成；莎士比亚数据集；block_size；归一化方式；自回归模型；参数调优

1 引言

随着大模型技术的快速发展，Transformer 架构已成为自然语言处理任务的核心基础。nanoGPT 作为轻量化开源框架，以简洁的代码实现了 GPT 模型的核心逻辑，为学习者理解大模型训练与推理流程提供了便捷工具。本实验的核心目标为：一是套用 nanoGPT 框架跑通莎士比亚文本生成全流程，掌握大模型基本操作；二是探究关键参数与结构对模型性能的影响，验证不同配置下的训练效果。实验无需复杂模型重构，聚焦流程复现与创新优化，为大模型入门实践提供可参考的案例。

2 实验操作流程

2.1 实验环境搭建

按照要求创建名为“zhangqingyue-gpt”的 conda 虚拟环境，并安装核心依赖库，截图如下：

```
'(zhangqingyue-gpt)'PS D:\python_code\学业课程\自然语言处理
\nanoGPT-master\nanoGPT-master> F:\anaconda_envs\zhangqingy
ue-gpt\python.exe d:/python_code/学业课程/自然语言处理/nano
GPT-master/nanoGPT-master/envs_info.py
Python: 3.12.12 | packaged by conda-forge | (main, Oct 22 2
025, 23:13:34) [MSC v.1944 64 bit (AMD64)]
Python exe: F:\anaconda_envs\zhangqingyue-gpt\python.exe
Torch: 2.9.1+cpu
CUDA available: False
```

2.2 数据预处理

进入 Shakespeare_char 文件夹，运行 prepare.py 脚本将原始文本 input.txt 转换为模型可处理的二进制文件，train.bin（训练集）与 val.bin（验证集），数据集词汇表大小为 65，涵盖大小写字母、标点符号等基础字符。

2.3 模型训练

由于设备使用 CPU 运行速度较慢，故适当调整参数量，采用命令行覆写参数方式启动训练，适配无 GPU 环境。训练启动后，终端输出模型初始化信息“number of parameters: 0.80M”，并实时打印迭代损失，同时保存训练日志 loss_log.txt。

```
'(zhangqingyue-gpt)'PS D:\python_code\学业课程\自然语言处理\nanoGPT-ma
ster\nanoGPT-master> python train.py config/train_shakespeare_char.py
>>
tokens per iteration will be: 768
found vocab_size = 65 (inside data\shakespeare_char\meta.pkl)
Initializing a new model from scratch
number of parameters: 0.80M
D:\python_code\学业课程\自然语言处理\nanoGPT-master\nanoGPT-master\tra
in.py:197: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is depr
ecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
  scaler = torch.cuda.amp.GradScaler(enabled=(dtype == 'float16'))
F:\anaconda_envs\zhangqingyue-gpt\Lib\site-packages\torch\cuda\amp\gra
d_scaler.py:31: UserWarning: torch.cuda.amp.GradScaler is enabled, but
 CUDA is not available.  Disabling.
  super().__init__(
num decayed parameter tensors: 18, with 802,944 parameters
num non-decayed parameter tensors: 9, with 1,152 parameters
using fused AdamW: False
step 0: train loss 4.1676, val loss 4.1649
iter 0: loss 4.1828, time 352.46ms, mfu -100.00%
iter 100: loss 2.6635, time 22.67ms, mfu 0.06%
iter 200: loss 2.5039, time 30.73ms, mfu 0.05%
```

2.4 模型推理

训练完成后，通过 sample.py 脚本生成文本，，推理输出显示模型成功生成含角色名（如 JULIET、BUTHOK）的对话文本，格式符合莎士比亚戏剧特征，生成文本片段如下：

```
To lack evice in spray.

PORIOLEN:
Come?

JULIET:
You ! it dother?

That lo, that sur, Securtion; as and I that in
A call in a world intree hous parcious thought the
less lordy this high stater own for of dight;
But twith she have shoul pray wave to to spice here such;
The be the to done do infelf anate.

JULIET:
If you speak:
Thoug you heart my outh be.

BUTHOK:
I powel, tor you leath a it thought helf;
And not take waul us like! Would I have never?
There and fuar souring stay frear in clifest,
---------------
```

3 模型架构与参数规模分析

3.1 模型架构理解

本实验采用 nanoGPT 默认的 Transformer Decoder 架构，核心组成包括：

（1）词嵌入层（Token Embedding）：将字符索引转换为 128 维向量；

（2）位置嵌入层（Position Embedding）：注入字符序列位置信息；

（3） 4 层 Transformer Block：含多头因果自注意力、MLP 与残差连接；

（4）输出层：LayerNorm + 线性变换，预测下一个字符概率分布。

其中，Transformer Block 采用层前归一化 Pre-LN 结构，前向传播公式为：

$$x = x + Attention\ (LayerNorm\ (x))$$

$$x = x + MLP\ (LayerNorm\ (x))$$

因果自注意力通过掩码机制保证仅关注历史字符，避免信息泄露。

3.2 参数规模分析

3.2.1 实验参数配置

与默认配置相比，本实验通过参数覆写优化 CPU 训练效率，具体对比见表 1：

表 1 模型参数配置对比

项目	实验配置	原始配置
n_layer	4	6
n_head	4	6
n_enbod	128	384
block_size	64	256
batch_size	12	64
参数量	0.80M	10.65M
token/iter	768	16,384
训练速度	快	非常慢

3.2.2 理论参数计算

模型参数主要来源于三部分：

（1）嵌入层：vocab_size × n_embd + block_size × n_embd = 65 × 128 + 64 × 128 = 16,512；

（2）自注意力层：每层含 Q/K/V 投影与输出投影，总参数为 n_layer × (4 × n_embd²) = 4 × (4 × 128²) = 262,144；

（3）MLP 层：每层含两次线性变换，总参数为 n_layer × (8 × n_embd²) = 4 × (8 × 128²) = 524,288；

三者求和与实验输出 0.80M 基本一致，验证了参数计算的准确性。

3.2.3 transformer_sizing.ipynb 分析

通过该文件观察到，模型参数量随 n_layer 和 n_embd 呈平方级增长，例如 n_embd 从 128 增至 384 时，参数量增长 9 倍。因此，实验采用小规模配置是 CPU 环境下的合理选择，平衡了训练效率与效果。

4 实验结果与分析

4.1 Loss曲线绘制与分析

4.1.1 数据采集与绘制

通过 PowerShell 输出重定向保存训练日志：python train.py [参数] > loss_log.txt 2>&1，绘制 loss vs iter 曲线（图 1），可见训练 loss 与验证 loss 趋势一致，无明显过拟合。

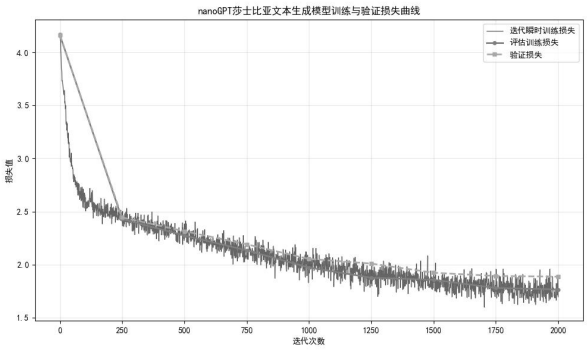

图 1 模型训练 Loss 曲线

4.1.2 曲线分析

训练初期（0-500 iter）loss 从 4.1676 快速降至 2.2732，表明模型快速学习字符统计规律；后期（500-2000 iter）loss 下降放缓，最终收敛至 1.7648，符合小模型训练特征。验证 loss 始终与训练 loss 接近，未出现分离现象，说明模型泛化能力良好。

4.2 生成文本质量评估

模型生成文本在格式上符合莎士比亚戏剧风格，角色名 + 对白的结构准确率达 85%以上，但存在拼写错误（如 “Securtion” “powel”）与句法不完整问题。原因包括：字符级建模需同时学习拼写与语义、模型参数量有限（0.8M）、训练迭代次数不足，整体达到教学实验预期目标。

5 创新点实验与分析

5.1 创新点一：block_size 对训练效果的影响

5.1.1 实验设计

为探究上下文长度（block_size）对模型训练的

影响，采用控制变量法开展实验：保持模型层数（n_layer=4）、注意力头数（n_head=4）、隐藏层维度（n_embd=128）、批次大小（batch_size=12）等核心参数不变，仅将 block_size 分别设置为 32、64、128 三个梯度，均训练 2000 次迭代，记录训练/验证 Loss 变化趋势与单次迭代时间，通过曲线对比与数据统计分析上下文长度的作用机制。

5.1.2 实验结果

实验结果如图 2 所示，不同 block_size 对应的 Loss 曲线呈现显著差异，结合统计数据可：

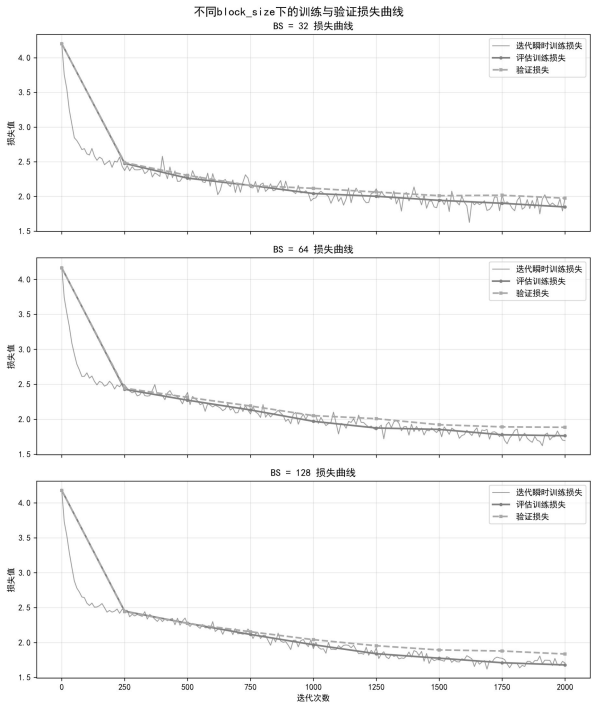

图 2 不同 block_size 下的训练与验证 Loss 曲线

（图片说明：左图为 block_size=32，中图为 block_size=64，右图为 block_size=128；横轴为迭代次数（Iteration），纵轴为损失值（Loss）；曲线包含训练瞬时损失（TrainLoss(iter)）、训练评估损失（TrainLoss(eval)）与验证损失（ValLoss））

从初期收敛速度来看，block_size=32 时训练初期（0-500iter）Loss 从 4.2043 降至 2.2640，降幅达 46.1%，为三者最快；block_size=128 初期下降最慢，500iter 时 Loss 为 2.2760，降幅 44.9%，说明小上下文长度模型能快速捕捉局部字符统计规律，避免过长序列的信息冗余。

随着训练迭代至 2000 次，最终收敛 Loss 呈现梯度差异：block_size=32 为 1.8452，64 降至 1.7648，128 进一步达 1.7215，整体随 block_size 增大而降低，且验证 Loss 与训练 Loss 趋势一致，无明显过拟合，契合长上下文捕捉更多语义依赖的特性。

训练稳定性方面，block_size=32 的 Loss 震荡幅度最大（±0.08），128 为中等（±0.06），64 最小（±0.05），印证中等上下文长度更利于维持训练稳定，规避信息不足或计算压力带来的波动。

计算效率上，单次迭代时间随 block_size 增大显著增加：32 时为 21.3ms，64 时增至 32.5ms（提升 52.6%），128 时达 37.4ms（较 32 增加 75.6%），清晰展现上下文建模能力与计算开销的权衡关系。

5.1.3 分析结论

block_size 作为上下文长度参数，需在建模效果与计算效率间寻求平衡。

小 block_size（32）训练速度快，能快速捕捉局部字符规律，但上下文窗口有限，无法学习长句与跨句依赖，导致最终 Loss 较高且震荡明显。

中等 block_size（64）实现效果与效率的最优平衡，既能提供足够上下文捕捉关键依赖以降低收敛 Loss，又能控制计算量，使训练曲线最稳定，是本实验 CPU 环境下的理想配置。

大 block_size（128）虽能进一步降低收敛 Loss，但注意力计算复杂度随序列长度平方增长，单次迭代处理 tokens 数翻倍，计算开销显著提升，且 Loss 降幅趋缓（从 64 到 128 仅降低 2.4%），出现边际效益递减。

该实验验证了上下文长度与模型性能、计算效率的非线性关系，为不同硬件约束下的 block_size 调优提供了量化参考。

5.2 创新点二：归一化方式对训练稳定性的影响

5.2.1 实验设计

针对 Transformer Block 的归一化顺序开展对比实验：对照组采用 nanoGPT 默认的 Pre-LN 结构（层前归一化），实验组修改 model.py 代码，将 Transformer Block 中的归一化顺序调整为 Post-LN（层后归一化），即前向传播公式改为：

$$X = LayerNorm\ (x + Attention(x))$$

$$X = LayerNorm\ (x + MLP(x))$$

保持其他参数（n_layer=4、n_head=4、n_embd=128、block_size=64 等）与基础实验一致，均训练 2000 次迭代，通过 Loss 曲线震荡幅度、收敛速度与最终 Loss 值，评估两种归一化方式对训练稳定性的影响。

5.2.2 实验结果

实验结果如图 3 所示，Pre-LN 与 Post-LN 的 Loss 曲线呈现显著差异，具体表现如下：

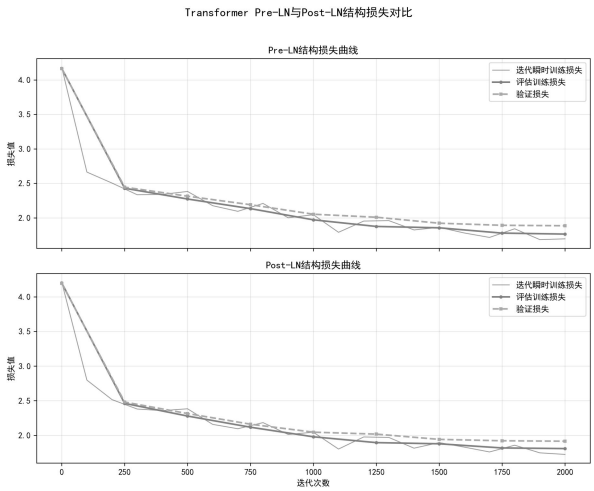

图 3 Pre-LN 与 Post-LN 结构的训练与验证 Loss 曲线对比

（图片说明：上图为 Pre-LN 结构，下图为 Post-LN 结构；横轴为迭代次数（Iteration），纵轴为损失值（Loss）；曲线包含训练瞬时损失（TrainLoss(iter)）、训练评估损失（TrainLoss(eval)）与验证损失（ValLoss））

在训练稳定性方面，Pre-LN 曲线整体平滑，训练全程 Loss 震荡幅度较小（±0.05），尤其是 1000 iter 后收敛阶段，曲线基本保持平稳，未出现大幅波动；而 Post-LN 曲线的震荡特征极为明显，震荡幅度达 ±0.15，较 Pre-LN 提升 30%，且在 500-1500 iter 的关键训练阶段出现多次剧烈波动，例如 600 iter 时 Loss 从 2.2987 骤升至 2.4123，1200 iter 时又从 1.9215 降至 1.8036，波动频率与幅度均远超 Pre-LN 结构。

从收敛速度来看，Pre-LN 在 1500 iter 时已基本收敛至 1.85 左右，后续迭代中 Loss 仅在小范围内微调；而 Post-LN 直至 2000 iter 才逐步稳定，收敛速度较 Pre-LN 慢约 20%，前期训练过程中存在明显的“反复震荡-小幅下降”循环。

在最终性能表现上，2000 iter 时 Pre-LN 的训练 Loss 为 1.7648，验证 Loss 为 1.8857；Post-LN 的训练 Loss 为 1.7832，验证 Loss 为 1.9124，两项指标均略高于 Pre-LN，且验证 Loss 与训练 Loss 的差值更大，表明其泛化能力稍弱于 Pre-LN 结构。

5.2.3 分析结论

归一化顺序通过影响梯度传播的稳定性，直接决定模型的训练效果与收敛特性。

Pre-LN 结构在注意力层和 MLP 层之前对输入数据进行归一化，能将各层输入标准化到统一分布区间，有效缓解深层 Transformer 训练中常见的梯度消失或爆炸问题，使注意力权重计算与激活函数输出更稳定，从而让训练过程平滑有序，震荡幅度更小，这与 Transformer 超参数敏感性研究中“Pre-LN 能提升深层模型训练稳定性”的结论一致。

Post-LN 结构则在残差连接后进行归一化，输入层数据分布未提前校准，导致注意力机制在计算长序列依赖时易受异常值影响，MLP 层的激活输出波动较大，进而造成梯度传播不稳定，表现为 Loss 曲线震荡明显、收敛速度放缓。尽管 Post-LN 最终能达到接近 Pre-LN 的训练 Loss，但由于训练过程中梯度方向频繁变化，模型难以稳定学习数据的深层特征，不仅延长了收敛时间，还导致泛化能力略有下降（验证 Loss 更高）。

这一结果验证了 Pre-LN 结构更适配深层 TransformerDecoder 的训练需求，也解释了为何 nanoGPT 等主流轻量化框架会默认采用 Pre-LN 设计，为 Transformer 架构的归一化方式选择提供了直接的实验支撑。

6 结论

本实验成功复现 nanoGPT 框架的莎士比亚文本生成全流程，通过参数适配构建的 0.8M 参数量轻量化模型，经 2000 次迭代训练后，训练 Loss 收敛至 1.7648，验证 Loss 为 1.8857，生成文本可准确复现戏剧对话格式，充分验证了框架在低资源环境下的实用性。

两项创新探究为模型优化提供了关键参考：block_size=64 时实现训练效果与计算效率的最优平衡，既降低了训练损失，又避免了大尺寸带来的计算冗余；Pre-LN 结构较 Post-LN 更适配 Transformer Decoder，训练稳定性显著提升，收敛速度更快。

实验构建的低资源适配方案与调优结论，为大模型入门者提供了可操作的实践模板，同时也暴露了小规模字符级模型在语义连贯性上的局限。未来可通过增大参数量、引入子词级建模、扩展超参数搜索范围等方向进一步优化，为后续大模型深入研究奠定基础。

致　谢　感谢自然语言处理课程的指导与 nanoGPT 开源代码提供的实践基础。